

Task 1: sales table(20-25 rows)

id, product_name, price

sum,min,max,avg,count

The screenshot shows the DBeaver SQL editor with a script to create a 'sales' table and insert 25 rows of data. The table has columns 'id', 'product_name', and 'price'. The 'product_name' column is defined as VARCHAR(100) and the 'price' column as INT.

```
1 # create table sales {
2
3 # id INT PRIMARY KEY,
4 # product_name VARCHAR(100),
5 # price INT
6 # }
7
8 # INSERT INTO sales VALUES (1, 'Apple', 10000);
9 # INSERT INTO sales VALUES (2, 'Banana', 500);
10 # INSERT INTO sales VALUES (3, 'Orange', 1200);
11 # INSERT INTO sales VALUES (4, 'Mango', 15000);
12 # INSERT INTO sales VALUES (5, 'Pineapple', 2000);
13 # INSERT INTO sales VALUES (6, 'Grapes', 1800);
14 # INSERT INTO sales VALUES (7, 'Watermelon', 15000);
15 # INSERT INTO sales VALUES (8, 'Cantaloupe', 10000);
16 # INSERT INTO sales VALUES (9, 'Kiwi', 300);
17 # INSERT INTO sales VALUES (10, 'Lemon', 1000);
18 # INSERT INTO sales VALUES (11, 'Lime', 800);
19 # INSERT INTO sales VALUES (12, 'Peach', 1200);
20 # INSERT INTO sales VALUES (13, 'Plum', 900);
21 # INSERT INTO sales VALUES (14, 'Cherry', 700);
22 # INSERT INTO sales VALUES (15, 'Strawberry', 600);
23 # INSERT INTO sales VALUES (16, 'Raspberry', 500);
24 # INSERT INTO sales VALUES (17, 'Blackberry', 400);
25 # INSERT INTO sales VALUES (18, 'Blueberry', 300);
26 # INSERT INTO sales VALUES (19, 'Elderberry', 200);
27 # INSERT INTO sales VALUES (20, 'Huckleberry', 100);
28 # INSERT INTO sales VALUES (21, 'Gooseberry', 100);
29 # INSERT INTO sales VALUES (22, 'Currant', 100);
30 # INSERT INTO sales VALUES (23, 'Rhubarb', 100);
31 # INSERT INTO sales VALUES (24, 'Kiwifruit', 100);
32 # INSERT INTO sales VALUES (25, 'Pomegranate', 100);
```

The screenshot shows the DBeaver SQL editor with a script to run aggregate queries on the 'sales' table. The queries calculate the total price, minimum price, maximum price, average price, and total number of products.

```
33 # SELECT SUM(price) AS total_sales
34 # FROM sales;
35
36 # SELECT MIN(price) AS minimum_price
37 # FROM sales;
38
39 # SELECT MAX(price) AS maximum_price
40 # FROM sales;
41
42 # SELECT AVG(price) AS average_price
43 # FROM sales;
44
45 # SELECT COUNT(*) AS total_products
46 # FROM sales;
47
48 # SELECT
49 #   SUM(price) AS total_price,
50 #   MIN(price) AS minimum_price,
51 #   MAX(price) AS maximum_price,
52 #   AVG(price) AS average_price,
53 #   COUNT(*) AS total_products
54 # FROM sales;
```

The results of the queries are displayed in a table at the bottom of the editor:

total_price	minimum_price	maximum_price	average_price	total_products
304400	100	15000	12176.0000	25

Mini project: Amazon data(ecommerce analysis):

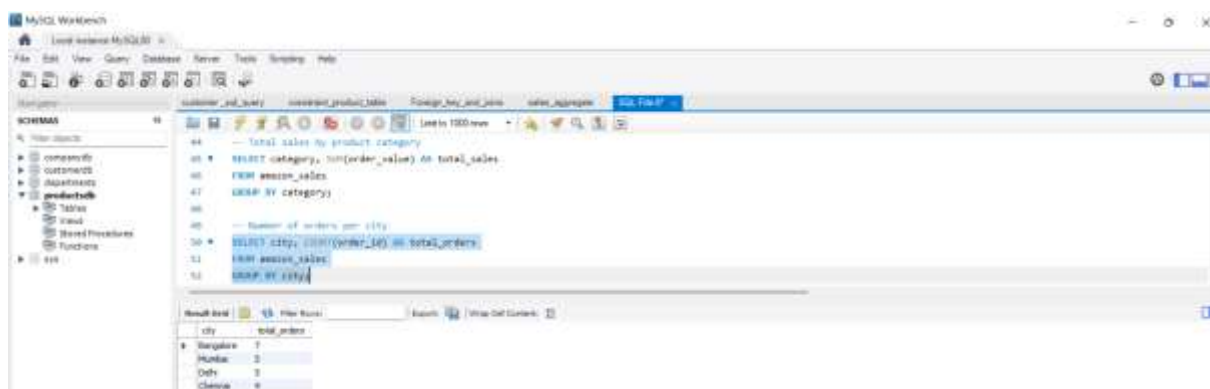
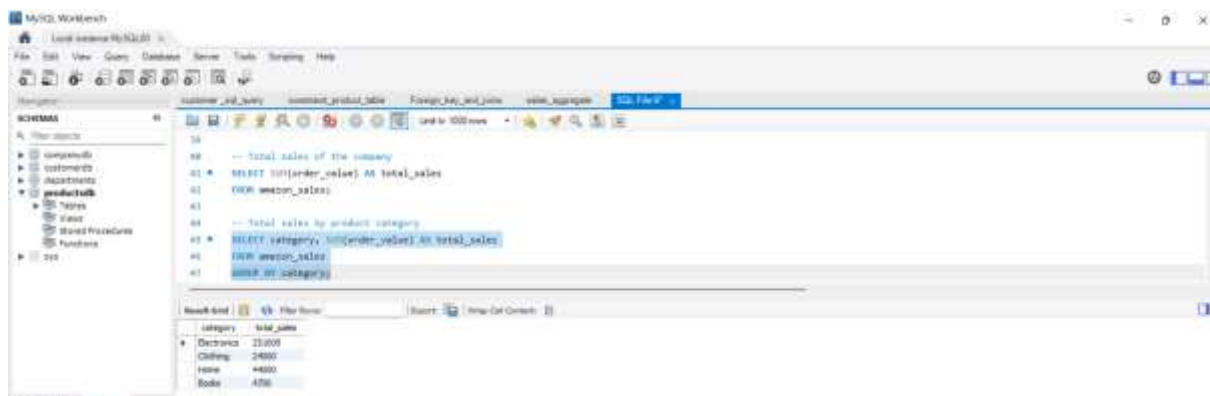
- 1.total sales of the company
- 2.total sales by product category
- 3.number of orders per city
- 4.average order value per customer
- 5.highest spending customer
- 6.maximum order value in each category

7.cities with sales greater than 50000

8.number of customers in each city

9.category with most orders

10.monthly sales



MySQL Workbench

Local instance MySQL5.7

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

- MySQL database
 - customers
 - customers
 - departments
 - products
 - Tables
 - Views
 - Stored Procedures
 - Functions
 - etc

Administration Schemas Information

No object selected

customer_id, sum, constant, product, table, Foreign key, and joins, view, aggregates, SQL File

10 SELECT city, COUNT(order_id) AS total_orders

11 FROM amazon_sales

12 GROUP BY city

13

14 -- Average order value per customer

15 SELECT customer_id, SUM(order_value) AS avg_order_value

16 FROM amazon_sales

17 GROUP BY customer_id

Result Grid

customer_id	avg_order_value
101	1100.0000
102	980.0000
103	1000.0000
104	900.0000
105	4200.0000
106	4000.0000
107	1200.0000
108	800.0000
109	3000.0000
110	900.0000
111	2000.0000
112	7000.0000
113	3000.0000
114	8000.0000
115	4000.0000
116	3400.0000
117	3000.0000
118	9000.0000

Result 1 of 1

MySQL Workbench

Local instance MySQL5.7

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

- MySQL database
 - customers
 - customers
 - departments
 - products
 - Tables
 - Views
 - Stored Procedures
 - Functions
 - etc

Administration Schemas Information

No object selected

customer_id, sum, constant, product, table, Foreign key, and joins, view, aggregates, SQL File

17 GROUP BY customer_id

18

19 -- Highest spending customer

20 SELECT customer_name, SUM(order_value) AS total_spent

21 FROM amazon_sales

22 GROUP BY customer_name

23 ORDER BY total_spent DESC

24 LIMIT 1

Result Grid

customer_name	total_spent
John	4200

Result 1 of 1

MySQL Workbench

Local instance MySQL5.7

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

- MySQL database
 - customers
 - customers
 - departments
 - products
 - Tables
 - Views
 - Stored Procedures
 - Functions
 - etc

Administration Schemas Information

No object selected

customer_id, sum, constant, product, table, Foreign key, and joins, view, aggregates, SQL File

27 GROUP BY customer_name

28 ORDER BY total_spent DESC

29 LIMIT 1

30

31 -- Product order value by each category

32 SELECT category, SUM(order_value) AS sub_order_value

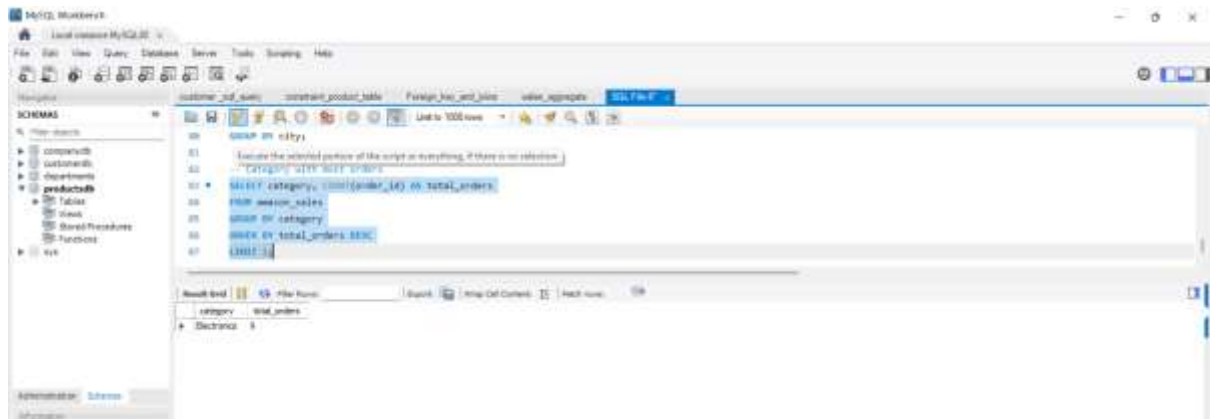
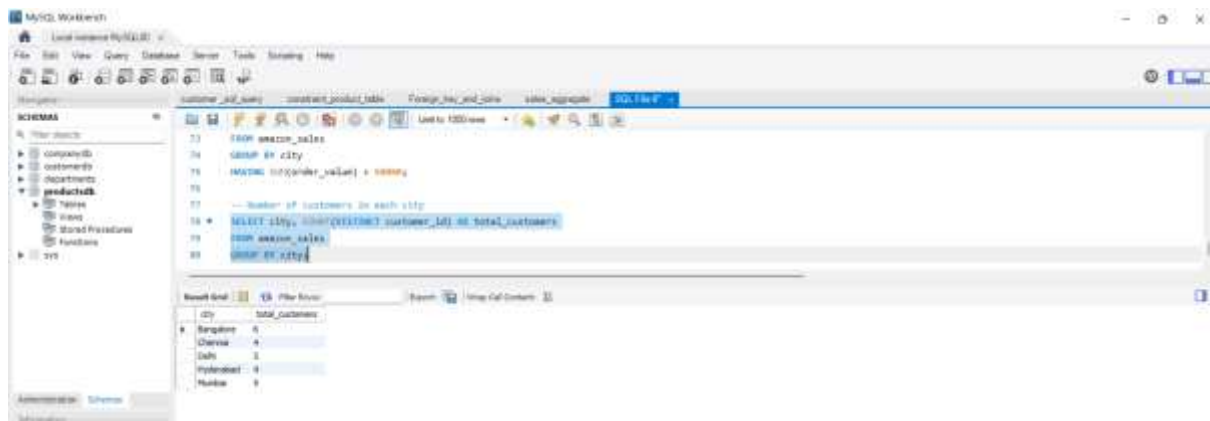
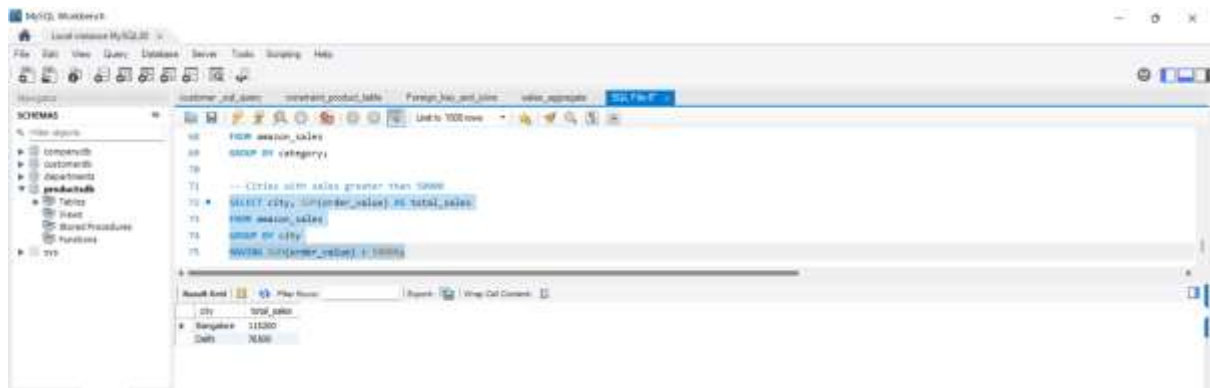
33 FROM amazon_sales

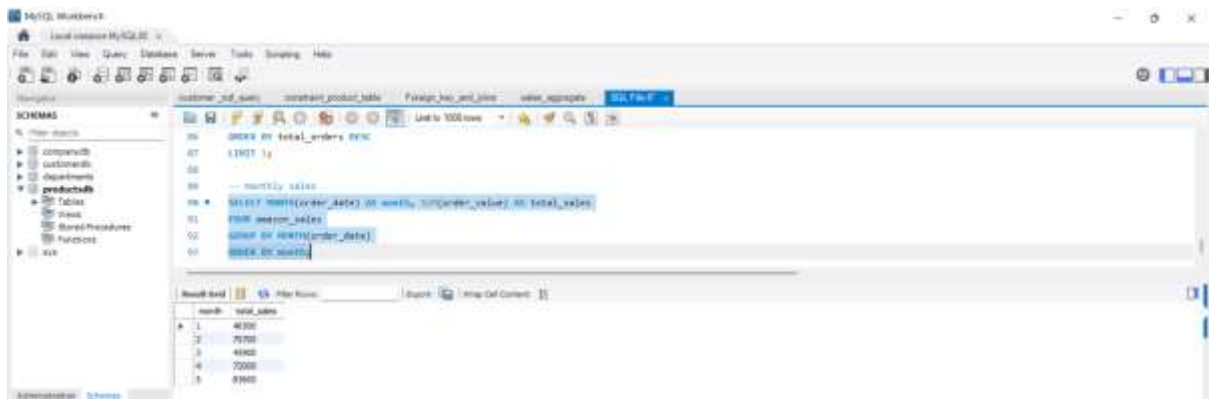
34 GROUP BY category

Result Grid

category	sub_order_value
Electronics	4200
Clothing	3000
Home	9000
Books	1500

Result 1 of 1

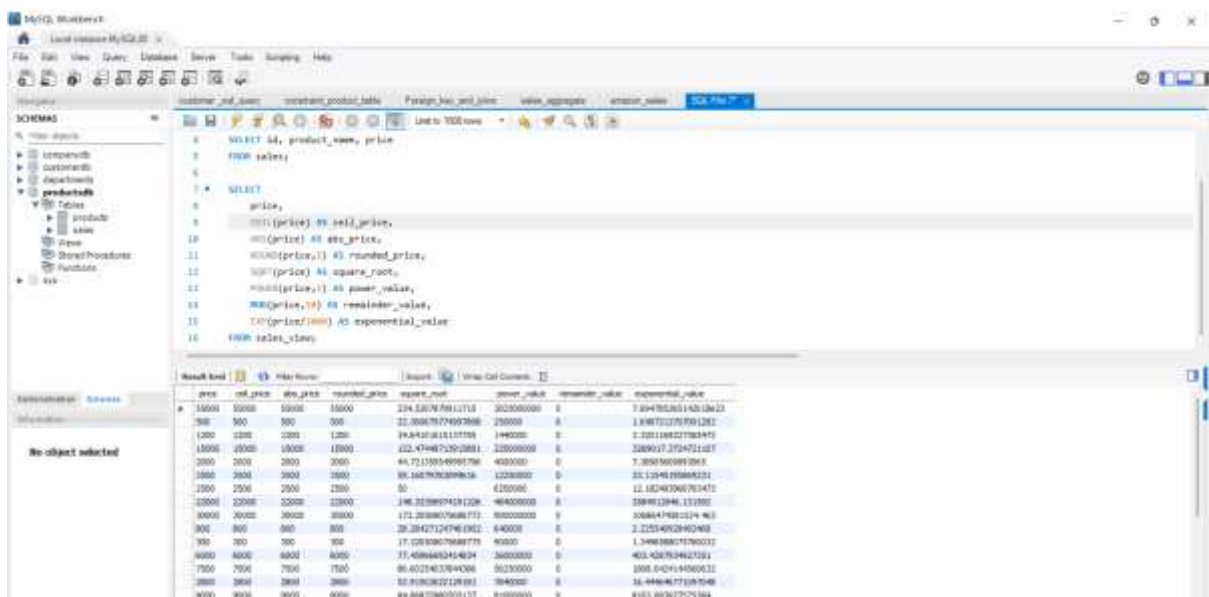




View:



Numerical functions:



Window Functions:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'products' selected. The main editor contains a SQL query using window functions to calculate row numbers and previous/next marks for products. The query is as follows:

```
13 SELECT  
14   (t1.marks + 70) AS  
15   (t1.marks + 70),  
16   (t1.marks + 70),  
17   ROW_NUMBER()  
18   OVER (ORDER BY marks DESC) AS rank_order,  
19   ROW_NUMBER()  
20   OVER (ORDER BY marks DESC) AS rank_order,  
21   ROW_NUMBER()  
22   OVER (ORDER BY marks DESC) AS rank_order,  
23   ROW_NUMBER()  
24   OVER (ORDER BY marks DESC) AS rank_order,  
25   ROW_NUMBER()  
26   OVER (ORDER BY marks DESC) AS rank_order
```

The 'Result Grid' at the bottom shows the output of the query. The columns are: rank_order, marks, rank_order, rank_order, rank_order, rank_order, rank_order. The data is as follows:

rank_order	marks	rank_order	rank_order	rank_order	rank_order	rank_order
1	95	1	1	1	1	1
2	92	2	2	2	2	2
3	90	3	3	3	3	3
4	88	4	4	4	4	4
5	85	5	5	5	5	5
6	82	6	6	6	6	6
7	80	7	7	7	7	7
8	78	8	8	8	8	8